



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №6
з дисципліни «Технології розроблення
програмного забезпечення»
Тема: «Патерни проектування»
Варіант «Mind-mapping software»

Виконав:

студент групи ІА-32

Бурдін Д. Є.

Перевірив:

Мягкий М. Ю.

Тема: Патерни проектування.

Мета: Вивчити структуру шаблонів «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator» та навчитися застосовувати їх в реалізації програмної системи.

Тема проєкту: Mind-mapping software

Патерни: Strategy, Prototype, Abstract Factory, Bridge, Composite, SOA

Опис: Візуальний додаток для складання "карт пам'яті" з можливістю роботи з декількома картами (у вкладках), автоматичного промальовування ліній, додавання вкладених файлів, картинок, відеофайлів (попередній перегляд); можливість додавання значків категорій / терміновості, обведення областей карти (поділ пунктирною лінією)

Зміст

Теоретичні відомості.....	2
Хід роботи	5
Діаграма класів патерну Abstract Factory	5
Код програми	5
Висновок	11
Питання до лабораторної роботи	11

Теоретичні відомості

Шаблон «Abstract Factory»

Призначення патерну: Шаблон «Абстрактна фабрика» використовується для створення сімейств об'єктів без вказівки їх конкретних класів [6]. Для цього виноситься загальний інтерфейс фабрики (AbstractFactory) і

створюються його реалізації для різних сімейств продуктів. Хорошим прикладом використання абстрактної фабрики є ADO.NET: існує загальний клас `DbProviderFactory`, здатний створювати об'єкти типів `DbConnection`, `DbDataReader`, `DbAdapter` та ін.; існують реалізації цих фабрик і об'єктів – `SqlProviderFactory`, `SqlConnection`, `SqlDataReader`, `SqlAdapter` і так далі. Відповідно, якщо додатку необхідно працювати з різними базами даних (чи потрібна така можливість), то досить використати базові реалізації (Db.) і підставити відповідну фабрику у момент ініціалізації фабрики (`Factory = new SqlProviderFactory()`).

Цей шаблон передусім структурує знання про схожі об'єкти (що називаються сімействами, як класи для доступу до БД) і створює можливість взаємозаміни різних сімейств (робота з Oracle ведеться також, як і робота з SQL Server). Проте, при використанні такої схеми украй незручно розширювати фабрику – для додавання нового методу у фабрику необхідно додати його в усіх фабриках і створити відповідні класи, що створюються цим методом.

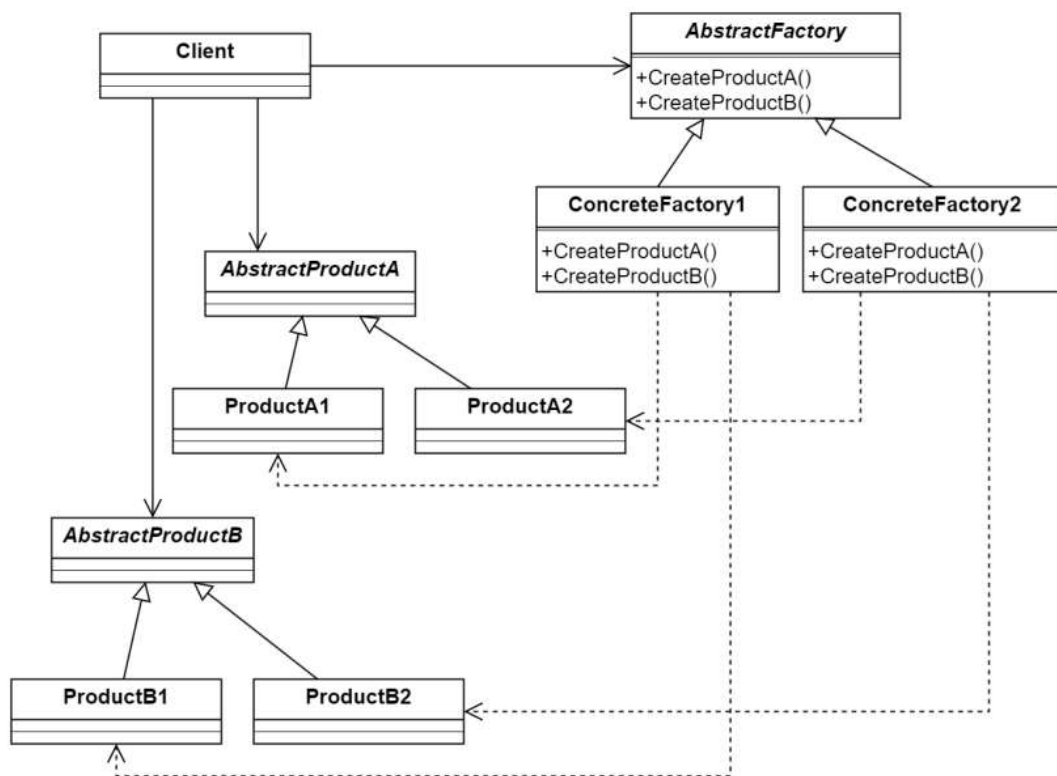


Рис. 1. Структура патерну «Абстрактна фабрика»

Проблема: Ви розробляєте для Roguelike гри модуль автоматичної генерації кімнат. Модуль генерації кімнат, в процесі генерації конкретної кімнати, створює стіни, двері, вікна, підлогу та меблі в кімнаті. Модуль повинен підтримувати генерацію кімнат у різних стилях, таких як стиль хайтек, модерн, 71 класичний офіс. Також критично є щоб всі елементи в одній згенерованій кімнаті відносилися до одного стилю.

Рішення: Для вирішення поставленої задачі дуже добре підходить використання патерну «Абстрактна фабрика».

Для кожного типу продукту створюємо свій інтерфейс продукту який об'являє функції доступні для використання з цим продуктом. Наприклад, можна виділити інтерфейси для стін, вікон, дверей, підлоги, а також окремі інтерфейси для меблів, такі як стіл, шафа, крісло та інші. Від кожного інтерфейсу наслідуюмо і реалізуємо продукти різних типів, наприклад, для стола: інтерфейс ITable та реалізації продуктів HighTechTable, ModernTable та інші.

Далі визначаємо інтерфейс для абстрактної фабрики, який буде містити методи для створення кожного типу продукту. Створюємо класи конкретних фабрик під кожен стиль: HighTechFabric, ModernFabric та інші. Кожна конкретна фабрика буде створювати всі типи продуктів, але всі вони будуть відноситися до одного стилю.

Далі в алгоритм генерації кімнати будемо передавати конкретну фабрику і алгоритм при створенні продуктів тільки через фабрику завжди буде отримувати продукти одного стилю і працювати з продуктами тільки через інтерфейс продукта.

Таким чином ми вирішуємо проблему узгодженості стилів всіх елементів в кімнаті, а за рахунок використання різних конкретних фабрик ми зможемо генерувати кімнати різних стилів. Якщо нам потрібно буде додати ще один стиль, то достатньо буде реалізувати нові дочірні класи для кожного елемента кімнати, а також нову конкретну фабрику під цей стиль.

Переваги та недоліки:

- + Спрощує створення об'єктів і код стає легшим для розуміння.
- + Об'єкти створені однією фабрикою добре узгоджуються один з одним і зменшується кількість помилок взаємодії між ними.
- + Відокремлення створення об'єктів від їх використання, за рахунок чого, код стає більш структурованим.
- + Додавання нових сімейств продуктів виконується без зміни існуючого коду.
- Збільшується складність коду, особливо для простих проєктів.
- Додавання нового типу продукту є складним і вимагає змін коду в багатьох місцях.

Хід роботи

Завдання

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Діаграма класів патерну Abstract Factory

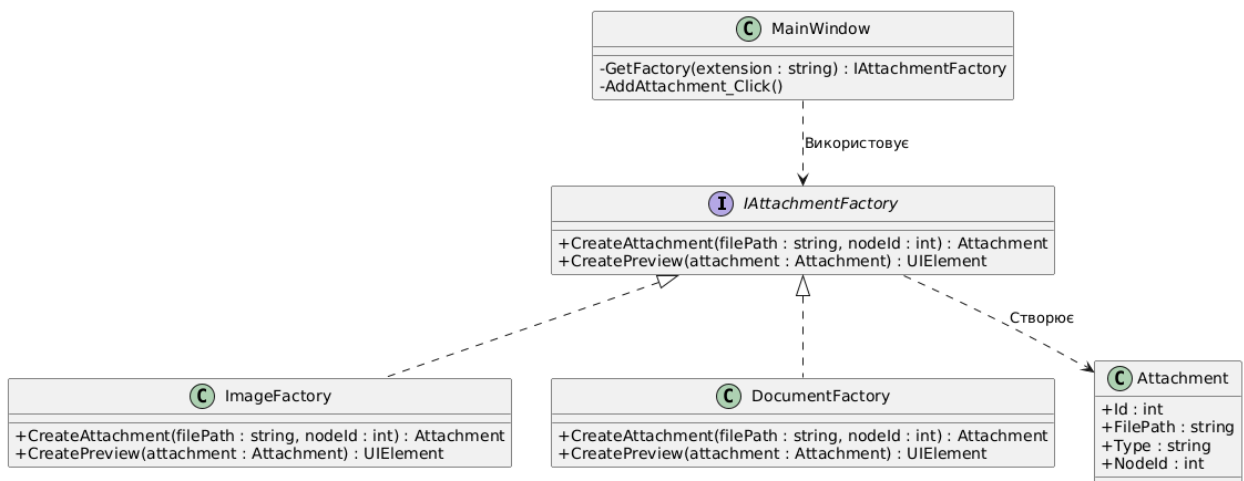


Рис. 2. Діаграма класів патерну Abstract Factory

Код програми

DocumentFactor

```
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using MindMapApp.Entities;
```

```
namespace MindMapApp.Factories
```

```
{
```

```
    public class DocumentFactory : IAttachmentFactory
```

```

{
    public Attachment CreateAttachment(string filePath, int nodeId)
    {
        return new Attachment
        {
            FilePath = filePath,
            FileName = System.IO.Path.GetFileName(filePath),
            Type = "FILE",
            NodeId = nodeId
        };
    }
}

public UIElement CreatePreview(Attachment attachment)
{
    var border = new Border
    {
        BorderBrush = Brushes.Gray,
        BorderThickness = new Thickness(1),
        CornerRadius = new CornerRadius(5),
        Padding = new Thickness(5),
        Margin = new Thickness(5),
        Background = Brushes.WhiteSmoke,
        Width = 100,
        Height = 100
    };

    var textBlock = new TextBlock
    {
        Text = "📄\n" + attachment.FileName,
    }
}

```

```

        TextWrapping = TextWrapping.Wrap,
        HorizontalAlignment = HorizontalAlignment.Center,
        VerticalAlignment = VerticalAlignment.Center,
        TextAlignment = TextAlignment.Center
    };

    border.Child = textBlock;
    return border;
}
}
}

```

IAttachmentFactory

```

using System.Windows;
using MindMapApp.Entities;
namespace MindMapApp.Factories
{
    public interface IAttachmentFactory
    {
        Attachment CreateAttachment(string filePath, int nodeId);
        UIElement CreatePreview(Attachment attachment);
    }
}

```

ImageFactory

```

using System;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media.Imaging;
using MindMapApp.Entities;

```

```
namespace MindMapApp.Factories
{
    public class ImageFactory : IAttachmentFactory
    {
        public Attachment CreateAttachment(string filePath, int nodeId)
        {
            return new Attachment
            {
                FilePath = filePath,
                FileName = System.IO.Path.GetFileName(filePath),
                Type = "IMAGE",
                NodeId = nodeId
            };
        }

        public UIElement CreatePreview(Attachment attachment)
        {
            try
            {
                var image = new Image();
                image.Width = 100;
                image.Height = 100;
                image.Margin = new Thickness(5);
                var bitmap = new BitmapImage();
                bitmap.BeginInit();
                bitmap.UriSource = new Uri(attachment.FilePath);
                bitmap.CacheOption = BitmapCacheOption.OnLoad;
                bitmap.EndInit();
                image.Source = bitmap;
                return image;
            }
            catch { }
        }
    }
}
```



```

    }
    catch
    {
        return new TextBlock { Text = "[Помилка зображення]", Foreground =
System.Windows.Media.Brushes.Red };
    }
}
}
}
}

```

EditNodeWindow.xaml (додано оновлення)

```

<ScrollView VerticalScrollBarVisibility="Auto" Height="80">
    <WrapPanel x:Name="AttachmentsPanel" Background="#F0F0F0"
MinHeight="50"/>
</ScrollView>

```

```

<Button Content="Додати файл..." Click="AddAttachment_Click"
Margin="0,5,0,15"/>

```

EditNodeWindow.xaml.cs (додано нові методи)

```

private void RenderAttachment(Attachment attachment)
{
    var factory = GetFactory(attachment.FilePath);
    var previewElement = factory.CreatePreview(attachment);
    var container = new StackPanel
    {
        Orientation = Orientation.Vertical,
        Margin = new Thickness(5),
        Width = 110
    };
    var deleteBtn = new Button
    {

```

```

        Content = "Видалити",
        FontSize = 10,
        Height = 20,
        Background = Brushes.IndianRed,
        Foreground = Brushes.White,
        Margin = new Thickness(0, 2, 0, 0),
        Cursor = System.Windows.Input.Cursors.Hand
    };
deleteBtn.Click += (s, e) =>
{
    AttachmentsPanel.Children.Remove(container);
    EditingNode.Attachments.Remove(attachment);
};
container.Children.Add(previewElement);
container.Children.Add(deleteBtn);
AttachmentsPanel.Children.Add(container);
}
private void AddAttachment_Click(object sender, RoutedEventArgs e)
{
    var dlg = new OpenFileDialog();
    if (dlg.ShowDialog() == true)
    {
        string path = dlg.FileName;
        var factory = GetFactory(path);
        var attachment = factory.CreateAttachment(path, EditingNode.Id);
        EditingNode.Attachments.Add(attachment);
        RenderAttachment(attachment);
    }
}
}

```

Висновок: Під час виконання лабораторної роботи було реалізовано патерн проєктування «Abstract Factory» що дозволило створити систему додавання файлових вкладень до вузлів на карті. Використання методу забезпечило інкапсуляцію логіки створення сімейств пов'язаних об'єктів — сутностей бази даних та відповідних їм візуальних елементів інтерфейсу

Питання до лабораторної роботи

1. Яке призначення шаблону «Абстрактна фабрика»?
Надати інтерфейс для створення сімейств взаємопов'язаних або залежних об'єктів без зазначення їхніх конкретних класів, що дозволяє клієнтському коду працювати з різними варіаціями продуктів, не знаючи про деталі їх реалізації.
2. Які класи входять в шаблон «Абстрактна фабрика», та яка між ними взаємодія?
Абстрактна фабрика, Конкретні фабрики, Абстрактні продукти, Конкретні продукти та Клієнт. Клієнт звертається до Абстрактної фабрики, яка делегує створення об'єктів відповідній Конкретній фабриці, повертаючи Клієнту набір сумісних продуктів.
3. Яке призначення шаблону «Фабричний метод»?
Шаблон визначає інтерфейс для створення об'єкта, але дозволяє підкласам вирішувати, екземпляр якого саме класу створювати. Це дозволяє делегувати логіку інстанціювання об'єктів класам-спадкоємцям, замінюючи пряме використання оператора new
4. Які класи входять в шаблон «Фабричний метод», та яка між ними взаємодія?
Творець, Конкретний Творець, Продукт та Конкретний Продукт. Творець викликає фабричний метод, а конкретна реалізація цього методу в підкласі (Конкретному Творці) повертає новий екземпляр Конкретного Продукту.
5. Чим відрізняється шаблон «Абстрактна фабрика» від «Фабричний метод»?
Абстрактна фабрика призначена для створення сімейств продуктів і реалізується через композицію, тоді як Фабричний метод створює один продукт і базується на успадкуванні.
6. Яке призначення шаблону «Знімок»?
Фіксувати та зберігати внутрішній стан об'єкта без порушення інкапсуляції, щоб пізніше можна було відновити об'єкт у цьому стані. Це є стандартним рішенням для реалізації функції скасування дій (Undo).
7. Які класи входять в шаблон «Знімок», та яка між ними взаємодія?

Творець, Знімок та Опікун. Опікун запитує створення знімка у Творця і зберігає його, не маючи доступу до вмісту, а за потреби передає знімок назад Творцю для відновлення стану.

8. Яке призначення шаблону «Декоратор»?

Дозволяє динамічно додавати об'єктам нову функціональність, загортаючи їх у корисні «обгортки». Це забезпечує гнучку альтернативу успадкуванню для розширення можливостей класу без зміни його структури.

9. Які класи входять в шаблон «Декоратор», та яка між ними взаємодія? Компонент, Конкретний компонент, Базовий декоратор та Конкретні декоратори. Декоратор містить посилання на об'єкт Компонента і делегує йому виконання роботи, додаючи свою поведінку до або після виклику.

10. Які є обмеження використання шаблону «декоратор»?

Основні обмеження це ускладнення коду через велику кількість дрібних класів, складність ініціалізації та конфігурування об'єкта, загорнутого в багато шарів декораторів, а також втрату ідентичності об'єкта, оскільки декоратор не є тим самим екземпляром, що й оригінальний об'єкт.